MAT4110 OBLIG 2

Zejing Wang

October 2024

1 The plan

I need to go to the webpage to download the pictures. They are the chessboard, jellyfish, and New York. I will use **Numpy** to apply some mathematical algorithms, and **matplotlib** will be used to plot the figures.
Next, I need to process this image. I will use **skimage.io** to read the picture, and **skimage.color** will use to change the color of the picture. I need to change the picture to grayscale.
And the picture represented by black and white, so is between 0 and 1, 0 is black, and 1 is white. **im1 = color.rgb2gray** is the code change the color of picture to grayscale.
The next step is **SVD** decomposition, I will use **np.linalg.svd**. this will use to break the matrix A into $A = USV^T$, where U is an orthogonal matrix include information of the pictures. S is a diagonal matrix, where is the singularvalue. If we have more singular value, this will include more informations. V^T is another orthogonal matrix.
When compressing an image, we retain only the largest r singular values and discard the smaller ones, reducing storage space and completing the compression process. This is the essentionly reduced **SVD**
I will use **plt.semilogy** to plot the log funtion.
At the last will I give some conclusion of advantages and disadvantages.

2 The code

This is the example code when $r = 1$, and we can change the value of r to different numbers.

```
import numpy as np
import matplotlib.pyplot as plt
from skimage import io, color
from skimage.util import img_as_float

im1 = io.imread('/Users/wangzejing/Downloads/board-157165_1280.png')
im2 = io.imread('/Users/wangzejing/Downloads/jellyfish-698521_1280.jpg'
    )
im3 = io.imread('/Users/wangzejing/Downloads/outdoors-5129182_1280.jpg'
    )


# Gray
im1_gray = color.rgb2gray(im1)
im2_gray = color.rgb2gray(im2)
```

```
14 im3_gray = color.rgb2gray(im3)
15
16 # change  gray picture to  between 0 and 1 floating number.
17 im1_gray = img_as_float(im1_gray)
18 im2_gray = img_as_float(im2_gray)
19 im3_gray = img_as_float(im3_gray)
20
21 # plot the gray.
22 plt.figure(figsize=(10, 10))
23
24 plt.subplot(1, 3, 1)
25 plt.imshow(im1_gray, cmap='gray')
26 plt.title('Chessboard')
27
28 plt.subplot(1, 3, 2)
29 plt.imshow(im2_gray, cmap='gray')
30 plt.title('Jellyfish')
31
32 plt.subplot(1, 3, 3)
33 plt.imshow(im3_gray, cmap='gray')
34 plt.title('New York')
35
36 plt.show()
37
38 def compress_image(image, r):
39     # SVd decomposition
40     U, S, Vt = np.linalg.svd(image, full_matrices=False)
41
42     #
43     U_r = U[:, :r]
44     S_r = np.diag(S[:r])
45     Vt_r = Vt[:r, :]
46
47     # compress the image
48     compressed_image = np.dot(U_r, np.dot(S_r, Vt_r))
49
50     return compressed_image, S
51
52 # Choose value of  r
53 r = 1 #
54
55 # Compress
56 compressed_im1, S1 = compress_image(im1_gray, r)
57 compressed_im2, S2 = compress_image(im2_gray, r)
58 compressed_im3, S3 = compress_image(im3_gray, r)
59
60 # plot
61 plt.figure(figsize=(10, 10))
62
63 plt.subplot(1, 3, 1)
64 plt.imshow(compressed_im1, cmap='gray')
65 plt.title(f'Chessboard (r={r})')
66
67 plt.subplot(1, 3, 2)
68 plt.imshow(compressed_im2, cmap='gray')
69 plt.title(f'Jellyfish (r={r})')
70
71 plt.subplot(1, 3, 3)
```

```
plt.imshow(compressed_im3, cmap='gray')
plt.title(f'New York (r={r})')

plt.show()

def compression_ratio(n, m, r):
    # After r * (n + m + 1)
    compressed_size = r * (n + m + 1)
    # before n * m
    original_size = n * m
    return original_size / compressed_size

# compute the ratio
n1, m1 = im1_gray.shape
n2, m2 = im2_gray.shape
n3, m3 = im3_gray.shape

ratio_im1 = compression_ratio(n1, m1, r)
ratio_im2 = compression_ratio(n2, m2, r)
ratio_im3 = compression_ratio(n3, m3, r)

print(f'Compression ratio (Chessboard): {ratio_im1:.2f}')
print(f'compression ratio (JellyFish): {ratio_im2:.2f}')
print(f'compression ratio (New York): {ratio_im3:.2f}')
# Plot the singular values using semilogy
plt.figure(figsize=(10, 6))

plt.semilogy(S1, label='Chessboard')
plt.semilogy(S2, label='Jellyfish')
plt.semilogy(S3, label='New York')

plt.title('Singular Values of Images')
plt.xlabel('Index')
plt.ylabel('Singular Value (log scale)')
plt.legend()
plt.grid(True)
plt.show()
```

3 Plot

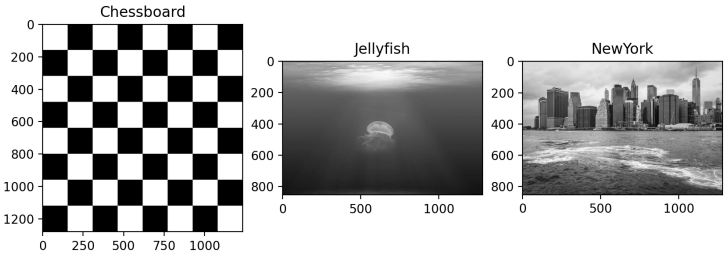

Figure 1: Changing the picture to grayscale .

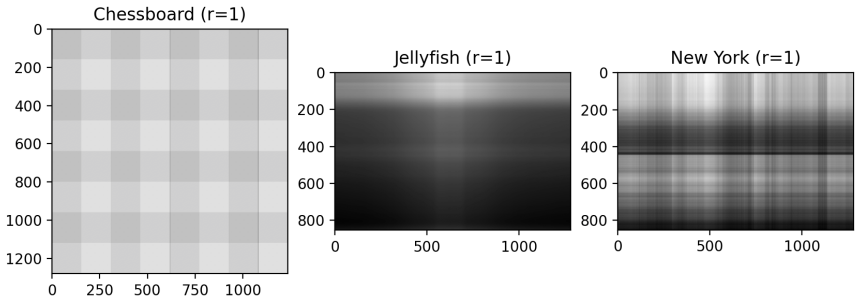

Figure 2: r=1 compressing the pictures

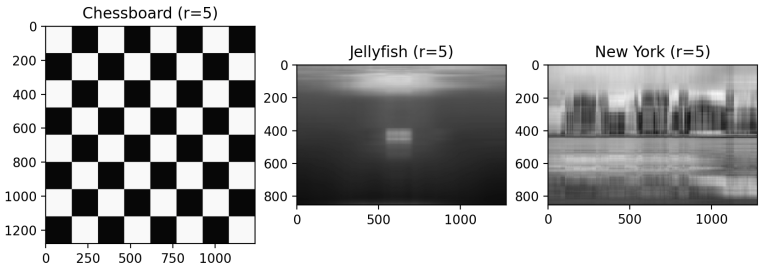

Figure 3: r=5, compressing the pictures

This first image is the grayscale image. Moreover, I will use different values of r to show the differences and how to preserve more information. It is obvious that there is a significant difference between the original image and the compressed images. When $r = 1$, A lot of information is lost, and the details are also lost. When we increase the value of r We can get an image with more information and more details if we use a relatively large r. There does not seem to be much difference before and after compression to the eye.

4 Compression Ratio

the compressing ratio for $r = 1$ is

```
Compression ratio (Chessboard): 628.56
Compression ratio (Jellyfish): 512.00
Compression ratio (New York): 511.64
```

The compressing ratio for $r = 100$ is

```
Compression ratio (Chessboard): 6.29
Compression ratio (Jellyfish): 5.12
Compression ratio (New York): 5.12
```

5 Plot log singular value

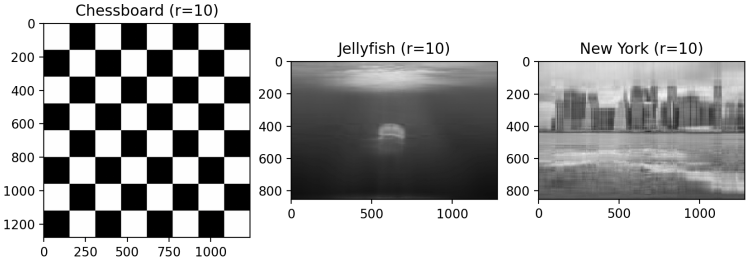

Figure 4: r=10 compressing the pictures

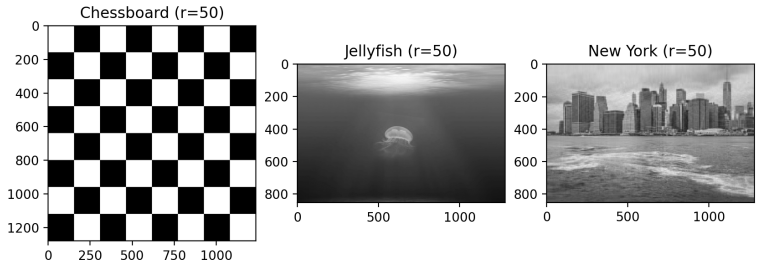

Figure 5: r=50 compressing the pictures

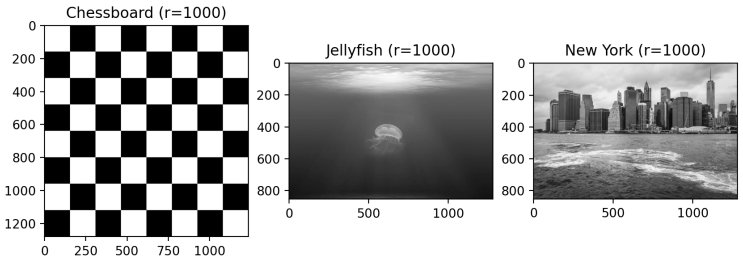

Figure 6: r=10000, compressing the pictures.

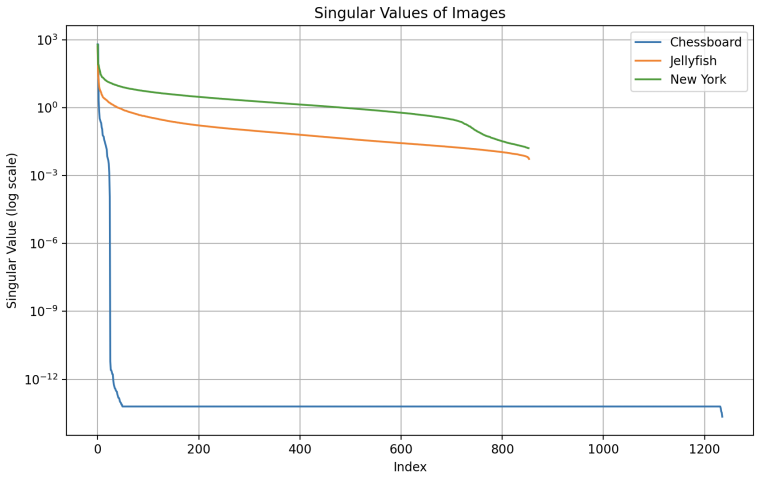

Figure 7: Use the semilogy to plot the log function.

```
# Plot the singular values using semilogy
plt.figure(figsize=(10, 6))

plt.semilogy(S1, label='Chessboard')
plt.semilogy(S2, label='Jellyfish')
plt.semilogy(S3, label='New York')

plt.title('Singular Values of Images')
plt.xlabel('Index')
plt.ylabel('Singular Value (log scale)')
plt.legend()
plt.grid(True)
plt.show()
```

I will use **plt.semilogy** to plot the log of the singular values, this will show the figure of log function, and the maximum singular value is on the figure.

6 Conclusion

The aim in this exercises is we want to retains most of the important visual information as much as possible. The tool is using the **SVD** decomposition. **SVD** have advantages and disadvantages.

6.1 Advantages

This algorithm is not very difficult, since we use Python, it's easy to use **numpy.linalg.svd skimage** to processing the image.
SVD decomposion do not need the matrix be non-singular, is available for all type of matrix , Especially for singular matrix. But some singularvalue will be 0 This means have high robustness.
Since $A = USV^T$ U V^T are both orthogonal. Its means they are more stability and we can retain the large singularvalue and discard the small singular value. This means we can proses the matrix which have high row and column and will have more numerical stability.

6.2 Disadvantages

By the plot after compressed , we know that if we want to retain more information, we must make the value of r very large. This will This will lead to bad consequences. For example, we let $r = 100$, this means we need find 100 singular value. if the first and the last one have too large difference. This will lose significant.
And since $A = USV^T$, this means we have high Computational complexity.This means if the matrix A have large many rows and columns, this will take long time. and this means **SVD** decomposition is not very useful to compress the image at real time, because this will take long time.

.